

Lab 9: KNN & Decision Tree

Objective:

- Explore and Preprocess the Telco Customer Churn Dataset.
- Implement and Evaluate KNN and Decision Tree Classification on the Telco Customer Churn Dataset.
- Perform Hyperparameter Tuning and Model Performance Comparison.

Software Requirements:

- (1) Python
- (2) scikit-learn
- (3) pandas
- (4) numpy
- (5) matplotlib

Dataset:

You may use the Telco Customer Churn dataset for conducting classification experiments with K-Nearest Neighbors (KNN) and Decision Tree models. This dataset contains customer demographic information, account details, subscribed services, and billing-related attributes from a telecommunications company. **The target variable is Churn, which indicates whether a customer has left the company, making it a binary classification task.** The dataset includes both numerical features (e.g., tenure, MonthlyCharges, TotalCharges) and categorical features (e.g., gender, Contract, PaymentMethod, InternetService), providing a good opportunity to explore and compare how KNN and Decision Tree models handle different types of attributes. Its moderate size and mixed feature types make it well-suited for hands-on practice in feature preprocessing, model training, hyperparameter tuning, and performance evaluation in a machine learning lab setting.

Lab Instructions

1. Import necessary libraries.

2. Follow the exercises below to build and evaluate your classification models using KNN and Decision Tree.

1) Data Loading and Exploration: (10pts)

- Use Pandas to load the dataset.
- Display the first 5 rows of the dataset.
- Explore the dataset using methods like head(), describe(), info().
- Visualize distributions of key features using histograms or boxplots.
- Draw feature correlation heatmap for the numerical features only.

2) Data Preprocessing: (20pts)

- Check whether any numeric-like columns need to be converted into numeric format before preprocessing (e.g., TotalCharges).
- Split the data into training (80%) and test (20%) sets.
- Convert categorical variables into numeric format using One-Hot Encoding by fitting the encoder on the training set only, then transforming the test set.
- Check for missing values. Handle missing values by fitting the imputer on the training set only, then applying it to the test set.
- Implement feature scaling by fitting the scaler on the training set only, then transforming the test set.

3) Model Building: (20pts)

- Building a classification model using KNN.
- Building a classification model using Decision Tree.
- Train both models on the training data.
- Predict the labels for the test set and store them in variables.

4) Model Evaluation: (20pts)

- Print out the accuracy_score and classification report for both models.
- Generate and visualize confusion matrices for both models.
- Draw the ROC curves for both models.
- Visualize the structure of your decision tree (at least the first three layers).

5) Hyperparameter Tuning: (15pts)

- Use GridSearchCV to find the best hyperparameters for both models.
- Output the best parameters and performance of two models

6) Cross Validation: (10pts)

- Perform 5-fold cross validation on two models and output the accuracy for each iteration and the average accuracy value for each model.

7) Performance Comparison: (5pts)

- Create a table or chart comparing the performances of KNN and Decision Tree.

Submission Requirement:

- 1) Complete sections 2 in a jupyter notebook file, name your file as “name_student number.ipynb” and submit it through “Lab 9 Programming Assignment” on iSpace.
- 2) Please submit the executed code.

Deadline: 11:59pm, Apr. 28, 2026; Plagiarism (cheating) = F